

PROVA FINAL Resultados para Yuske Eduardo Velit Arruda

Pontuação desta tentativa: 36 de 40

Enviado 7 jan em 19:16

Esta tentativa levou 8 minutos.

Resposta correta



Pergunta 1

4 / 4 pts

O ORM é uma técnica que pode ser utilizada em aplicações de qualquer porte. Essa técnica possibilita que a aplicação evolua de forma mais saudável e adiciona uma série de facilidades na utilização dos métodos pela camada de repositório.

Assinale a alternativa CORRETA a respeito dos ORMs:



O ORM é uma técnica que possibilita a abstração da implementação dos casos de uso. Com isso, os métodos de regras de negócio encapsulam as chamadas à APIs externas e fazem com que alterações no acesso à dados não afetem a regra de negócios da aplicação.



O ORM é uma técnica que possibilita a abstração da implementação dos controladores. Com isso, os métodos de endpoint encapsulam os caso de uso e fazem com que alterações na regra de negócio não afetem a camada de acesso a dados.



O ORM é uma técnica que possibilita a abstração da implementação de acesso a banco. Com isso, os métodos de acesso encapsulam as queries e fazem com que alterações no acesso à dados não afetem a regra de negócios da aplicação.



A técnica do ORM consiste em abstrair a instalação de dependências em uma única biblioteca. Sendo assim, caso haja uma alteração em qualquer dependência externa da aplicação, podemos fazer uma única alteração no package.json, uma vez que as instalações estão abstratidas.



O ORM traz uma complexidade muito grande para as aplicações, não sendo nunca recomendado para projetos de pequeno ou médio porte.

Resposta correta



Pergunta 2

4 / 4 pts

O padrão repositório é amplamente usado para separar a lógica de acesso ao banco da lógica de negócio. Isso tem uma série de impactos na aplicação no longo prazo. Sendo assim, assinale a alternativa correta sobre a manutenção de aplicações que utilizam o padrão repositório:

- ☐ Garante que apenas queries SQL sejam usadas.
- ☐ Elimina a necessidade de DTOs e serviços.
- ☒ Facilita a substituição do banco de dados sem grandes alterações na lógica da aplicação.
- ☐ Permite que o frontend acesse diretamente os dados.
- ☐ Substitui a necessidade de validações no controller.

Resposta correta



Pergunta 3

4 / 4 pts

O NestJS oferece recursos como decorators (`@Controller` , `@Injectable` , `@Module`) que ajudam a organizar melhor o código. Com isso, podemos ter de forma clara na implementação a representação da responsabilidade da classe.

Ao criar um `@Controller` , por que é necessário declarar o service no construtor da classe?

- ☒ Porque o NestJS utiliza injeção de dependência para fornecer instâncias únicas dos serviços, promovendo reuso e organização.
- ☐ Para que o controller atue como repositório da aplicação.
- ☐ Porque controllers não funcionam sem entidades.
- ☐ Porque o NestJS exige instanciar todas as classes manualmente.
- ☐ Para garantir que o controller se comporte como singleton.

Resposta correta



Pergunta 4

4 / 4 pts

A seguir, temos a implementação de uma entidade `Invoice` e seu repositório personalizado com métodos específicos para manipulação de faturas:

```
// invoice.entity.ts
@Entity()
export class Invoice {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  clientId: number;

  @Column({ default: false })
  paid: boolean;
```

```
@Column({ type: 'timestamp', nullable: true })
paidAt?: Date;
}

// invoice.repository.ts
@Injectable()
export class InvoiceRepository {
  constructor(
    @InjectRepository(Invoice)
    private readonly repo: Repository<Invoice>,
  ) {}

  async findUnpaidByClient(clientId: number): Promise<Invoice[]> {
    return this.repo.find({ where: { clientId, paid: false } });
  }

  async markAsPaid(id: number): Promise<Invoice> {
    const invoice = await this.repo.findOne({ where: { id } });
    if (!invoice) throw new NotFoundException();
    invoice.paid = true;
    invoice.paidAt = new Date();
    return this.repo.save(invoice);
  }
}
```

Em relação à implementação apresentada, qual alternativa expressa corretamente boas práticas e uso adequado do padrão Repository no NestJS?



A abordagem respeita a separação de responsabilidades, pois o repositório encapsula o acesso ao banco e expõe operações específicas sem violar regras de domínio.



O repositório está assumindo funções indevidas do service, pois trata estados e exceções que não pertencem à camada de persistência.



O uso de `@InjectRepository()` é inadequado para entidades com múltiplos campos.



A lógica de marcação de faturas como pagas deveria estar fora do repositório e delegada ao controller



A entidade deveria conter lógica interna de validação, como lançar exceções diretamente nos métodos `setPaid()`.

Resposta correta



Pergunta 5

4 / 4 pts

A Clean Architecture propõe que as regras de negócio fiquem totalmente desacopladas de frameworks, UIs e fontes de dados, o que favorece testabilidade e manutenção. Este tipo de

arquitetura possui práticas recomendadas que garantem que a aplicação continue escalável e robusta no decorrer do tempo e de acordo com que novas funcionalidades vão sendo adicionadas.

Assinale a alternativa que melhor descreve os principais pilares da arquitetura CLEAN:

- ☐ Transição de dados e chamadas livre entre as camadas, não importando a sua ordem.
- ☒ Separação de responsabilidades em camadas, com dependência unidirecional do externo para o interno.
- ☐ Redução de módulos e maior acoplamento com o banco de dados
- ☐ Encapsulamento das requisições websocket, uma vez que o modelo de APIs REST é pouco recomendado em arquiteturas CLEAN
- ☐ Prevenção à reutilização de código dentro de uma mesma camada, fazendo com que a duplicidade favoreça a robustez da aplicação

Resposta incorreta



Pergunta 6

0 / 4 pts

O sistema de módulos no Node.js permite dividir a aplicação em partes reutilizáveis, facilitando a manutenção e escalabilidade da aplicação. Com base no sistema de módulos, assinale a alternativa **VERDADEIRA**.

- ☐ É necessário compilar os módulos manualmente.
- ☐ Um módulo só pode ser importado uma única vez.
- ☒ Apenas funções podem ser exportadas como módulos.
- ☐ Qualquer valor (objeto, classe, função, etc.) pode ser encapsulado como módulo.
- ☐ O sistema de módulos impede o uso de classes.

Resposta correta



Pergunta 7

4 / 4 pts

A estrutura modular do NestJS permite que cada parte da aplicação fique isolada em seu próprio escopo caso necessário. Com base na estrutura de módulos e nos conceitos apresentados, responda: qual a principal vantagem da arquitetura baseada em módulos no NestJS?

- ☐ Torna o uso de TypeScript opcional.
- ☐ Substitui o uso de banco de dados.
- ☐ Permite múltiplos servidores rodando em paralelo.
- ☒ Garante a escalabilidade e técnica por meio da separação lógica de responsabilidades.
- ☐ Elimina a necessidade de usar services.

Resposta correta



Pergunta 8

4 / 4 pts

Na arquitetura Clean, a camada de casos de uso (use cases) é responsável por orquestrar a lógica de aplicação, coordenando entidades, repositórios e regras de negócio.

Considere a seguinte afirmação:

"Em uma aplicação com arquitetura limpa, a camada de caso de uso deve se comunicar com a camada de repositório para executar as ações necessárias, sem depender da camada de interface (ex: controllers)."

Assinale a alternativa correta em relação ao papel da camada de caso de uso:

- ☐ Casos de uso existem apenas em APIs REST, não sendo aplicáveis em sistemas de fila ou eventos.
- ☐ A camada de casos de uso deve conter lógica de roteamento e controle de autenticação.
- ☐ Os casos de uso são implementados dentro dos controllers para garantir controle total do fluxo de dados.
- ☐ Os casos de uso são responsáveis por emitir eventos e persistir dados diretamente, ignorando repositórios.

☒ Casos de uso centralizam regras de negócio da aplicação e interagem com interfaces de persistência e entidades, mantendo-se independentes da UI e da infraestrutura.

Resposta correta



Pergunta 9

4 / 4 pts

No desenvolvimento de aplicações baseadas em Micro Serviços, existem basicamente dois tipos de benefícios em termos estruturais: escala técnica e escalas organizacional.

Esses benefícios podem ir além da estruturação da aplicação e extrapolar os conceitos de linguagens de programação, banco de dados e servidores.

Com base nos benefícios apresentados pela arquitetura de Microserviços, assinale a alternativa **CORRETA**:



Os benefícios da criação de microserviços dizem respeito APENAS a parte organizacional da empresa, como: possibilidade de segregação de times onde cada time pode ficar independente e responsável por um serviço, facilidade na contratação de pessoas e etc.



Os benefícios da criação de microserviços dizem respeito APENAS aos quesitos técnicos da arquitetura, como: independência entre linguagens, possibilidade de múltiplos bancos de dados para cada serviço e etc.



A escala organizacional possibilita a segregação do desenvolvimento de serviços por times. Na escala técnica pensamos nos recursos de máquina, servidores e estratégias a nível de desenvolvimento.



A escala organizacional lista as possibilidades de orquestração de micro serviços através de Kubernetes. Na escala técnica pensamos nos recursos de máquina, servidores e estratégias a nível de desenvolvimento.



A escala técnica possibilita a segregação do desenvolvimento de serviços por times. Na escala organizacional pensamos nos recursos de máquina, servidores e estratégias a nível de desenvolvimento.

Resposta correta



Pergunta 10

4 / 4 pts

O NestJS abstrai o uso do Express, oferecendo uma arquitetura modular com suporte a injeção de dependências. Além disso, ele torna mais fácil a utilização de padrões arquiteturais conhecidos, pois a sua estrutura permite que possamos estruturar a aplicação de maneira escalável, favorecendo a manutenção.

Com base na estrutura do NestJs, qual elemento é responsável por receber requisições HTTP e repassá-las à camada de serviço?

- ☒ Controller
- ☐ DTO (Data Transfer Object)
- ☐ Interceptor
- ☐ Module
- ☐ Provider

Pontuação do teste: 36 de 40